1.补全程序，主要是前面的 3 个空和生成诗歌的一段代码。

已经在程序中补全。

2.解释一下 RNN ，LSTM，GRU 模型

三者都是处理序列数据的循环神经网络模型，核心思想是“当前时刻不仅看当前输入，还看历史状态”。

RNN 是最基础形式，状态更新简单：用上一步隐藏状态和当前输入算新状态。优点是结构简单；缺点是序列长时容易梯度消失/爆炸，难学到长期依赖。

LSTM 是在 RNN 上加入记忆单元和门控机制（输入门、遗忘门、输出门），通过门来决定写入什么、忘记什么、输出什么。它能更稳定地保留长期信息，长序列表现通常比普通 RNN 好，但参数更多、计算更重。

GRU 是 LSTM 的简化版，用更新门和重置门替代三门和独立细胞状态，结构更轻。它通常训练更快、参数更少，在很多任务上效果接近 LSTM，有时更好。

3.叙述一下 这个诗歌生成的过程。

本实验采用的是基于 LSTM 的字符级循环神经网络来做古诗生成。具体是两层 LSTM：输入为当前字的词向量，输出为“下一个字”的概率分布。

整体流程是：先对语料清洗并加起止符，把每首诗转成字索引序列，再构造监督样本 x 和 y，其中 y 是 x 右移一位；训练时把序列送入 LSTM，经全连接层和 LogSoftmax 得到每个位置的预测概率，用 NLLLoss 计算损失并反向传播更新参数，周期性保存模型。

生成阶段先给定开头字作为 begin_word，把当前已生成文本输入模型，取最后一个时间步预测概率最大的字（argmax）作为下一个字，并拼接回输入继续循环，直到生成结束符或达到最大长度。这个过程本质是自回归逐字生成。

4.生成诗歌 开头词汇是 “ 日、红 、山 、 夜 、 湖、 海 、 月 ” ，等词汇作为 begin word,把生成的截图放到报告里面。[pytorch 版本 需要放一张 训练时候的截图。]

```
******************************
epoch   29 batch number 344 loss is:  6.132923603057861
prediction [10, 7, 46, 44, 159, 0, 10, 357, 10, 22, 21, 1, 10, 8, 84, 474, 42, 0, 7, 78, 4, 54, 630, 1, 7, 183, 4, 4,
 43, 0, 7, 45, 4, 138, 35, 1, 10, 48, 12, 93, 327, 0, 6, 21, 8, 19, 28, 1, 7, 119, 47, 740, 42, 0, 6, 17, 34, 301, 69
2, 1, 99, 17, 9, 119, 74, 0, 69, 24, 10, 39, 6, 1, 3, 3]
b_y       [196, 330, 69, 44, 70, 0, 581, 451, 18, 714, 457, 1, 8, 169, 5, 24, 73, 0, 6, 40, 100, 54, 626, 1, 185, 11
, 32, 1148, 1148, 0, 249, 16, 63, 236, 236, 1, 118, 48, 168, 450, 378, 0, 6, 21, 101, 289, 615, 1, 22, 50, 259, 232,
252, 0, 33, 364, 117, 93, 594, 1, 233, 161, 88, 201, 943, 0, 86, 30, 43, 39, 632, 1, 3, 3]
******************************
epoch   29 batch number 345 loss is:  6.28184700012207
prediction [10, 18, 7, 327, 70, 0, 10, 8, 4, 30, 30, 1, 10, 6, 4, 10, 70, 0, 4, 49, 5, 14, 32, 1, 7, 18, 9, 7, 16, 0,
 22, 23, 273, 104, 12, 1, 10, 16, 9, 22, 16, 0, 4, 57, 47, 17, 70, 1, 4, 172, 10, 190, 45, 0, 4, 43, 10, 107, 5, 1, 1
5, 488, 30, 62, 43, 0, 0, 0, 9, 178, 4, 96, 4, 1, 1, 1, 3, 3]
b_y       [457, 152, 475, 147, 70, 0, 223, 8, 37, 313, 1117, 1, 302, 457, 231, 652, 1824, 0, 222, 704, 70, 147, 5, 1
, 457, 48, 84, 88, 16, 0, 201, 72, 1766, 372, 1209, 1, 88, 16, 112, 594, 93, 0, 12, 57, 56, 23, 79, 1, 149, 278, 80,
309, 2542, 0, 28, 175, 254, 217, 1113, 1, 271, 488, 450, 135, 43, 15, 42, 0, 12, 65, 564, 475, 540, 157, 29, 1, 3, 3]
******************************
epoch   29 batch number 346 loss is:  6.3919291496276855
prediction [10, 23, 179, 5, 159, 97, 57, 0, 4, 178, 94, 8, 10, 178, 35, 1, 4, 6, 4, 34, 4, 16, 6, 0, 10, 1125, 119, 8
70, 4, 97, 107, 1, 10, 30, 4, 115, 97, 149, 212, 0, 4, 131, 9, 177, 4, 26, 38, 1, 10, 5, 4, 0, 4, 30, 337, 0, 4, 7, 9
4, 43, 43, 416, 1, 1, 4, 39, 84, 35, 0, 0, 0, 0, 4, 62, 4, 57, 4, 6, 17, 1, 3, 3]
b_y       [188, 682, 804, 242, 143, 242, 70, 0, 10, 242, 297, 41, 10, 242, 100, 1, 342, 71, 191, 6, 240, 60, 154, 0,
 555, 85, 1335, 1335, 69, 97, 107, 1, 139, 284, 325, 196, 4, 149, 95, 0, 36, 11, 99, 73, 144, 37, 381, 1, 38, 5, 112,
 163, 1572, 241, 867, 0, 182, 7, 1076, 1076, 243, 134, 434, 1, 441, 70, 296, 457, 869, 108, 20, 0, 729, 131, 907, 117
, 66, 6, 539, 1, 3, 3]
******************************
epoch   29 batch number 347 loss is:  6.285831451416016
-----
```

```
(tf-env) E:\大学\课\深度学习\exercise\chap6_RNN\tangshi_for_pytorch>python main.py
error
inital  linear weight
一别不相问，此时何处归。
error
inital  linear weight
红尺发，不复黄金不可寻。
何事此时无此事，不妨收此为君行。
error
inital  linear weight
一夜一双思，千山万里通。
error
inital  linear weight
风光无处所，风物不成行。
error
inital  linear weight
一别不可问，一枝无一枝。
error
inital  linear weight
一别不可问，一枝无一枝。
error
inital  linear weight
一别不可问，一枝无一枝。
error
inital  linear weight
不是当时事，无人问此贤。
```